2021 IEEE/ACM 43rd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER) | 978-1-6654-0140-1/20/$31.00 ©2021 IEEE | DOI: 10.1109/ICSE-NIER52604.2021.00009

Validation Obligations: A Novel Approach to Check Compliance between Requirements and their Formal Specification

1st Atif Mashkoor
Johannes Kepler University
Linz, Austria
atif.mashkoor@jku.at

2nd Michael Leuschel
Heinrich Heine University
Düsseldorf, Germany
leuschel@hhu.de

3rd Alexander Egyed
Johannes Kepler University
Linz, Austria
alexander.egyed@jku.at

***Abstract*—Traditionally, practitioners use formal methods predominately for one half of the quality-assurance process: verification (do we build the software right?). The other half – validation (do we build the right software?) – has been given comparatively little attention. While verification is the core of refinement-based formal methods, where each new refinement step must preserve all properties of its abstract model, validation is usually postponed until the latest stages of the development, when models can be automatically executed. Thus mistakes in requirements or in their interpretation are caught too late: usually at the end of the development process. In this paper, we present a novel approach to check compliance between requirements and their formal refinement-based specification during the earlier stages of development. Our proposed approach – "validation obligations" – is based on the simple idea that both verification and validation are an integral part of all refinement steps of a system.**

***Index Terms*—formal methods, refinement, specification, validation obligations**

I. Introduction

In his 1993 report on "Formal Methods and the Certification of Critical Systems" [1], Rushby raised concerns that due to the unprovable nature of validation as compared to verification, the former one is to a large extent neglected by the formal methods community. While great advancements have been made in the verification area, which led to several industrial-strength environments for verification, such as RODIN [2], the validation challenge is still open [3]. It is not like that the formal methods community is oblivious to this important activity: we have methods like requirements state machine language (RSML) [4] and software cost reduction (SCR) [5], which are designed with an active consideration to validation. However, their contribution towards validation[1] of systems requirements is limited in the step-wise refinement process.

One of the main impediments for the diffusion of formal methods in the software development industry is the great difficulty to validate formal models before they are implemented. This impedes the implementation of a regular quality-assurance procedure. While the verification of a formal specification emphasizes on proving its internal consistency, validation addresses another important question entirely: whether the specification is an adequate representation of the system to be developed. Validation thus includes by definition an informal assessment: that consequences derived from the specification are in agreement with observations of a part of reality or with expectations of future users. Except for few cases (mostly numerical models), validation cannot be formalized by simply checking that "measures" on the real objects are equal to "predictions" of the model. In fact, in the case of a specification of a new system, the comparison is not with reality but with "wishes" of stakeholders.

Refinement-based formal methods, such as B [7] and Event-B [8], are highly significant for the development of (safety-critical) software systems. Refinement lets engineers structure models, verify them, and generate code by gradually adding details to the higher-level model while maintaining the already established formal properties. In this context, proof obligations (POs)[2] are crucial to ensure that models of a software system under construction are consistent and each refinement step, where a coarser-grained model is refined into a finer-grained model, preserves the properties described at the coarser-grained levels. POs also help when incrementally adapting a formal model, by pinpointing critical issues after a change of the model or the specification. However, POs do not guarantee that the "right" system has been developed, i.e., the development captures all stakeholders' requirements. This is the task of validation – a process which is often carried out in an unstructured manner using techniques such as animation, simulation, model checking, code review, or acceptance testing of the generated code.

The overall aim of this paper is to enhance the refinement-based formal software development process by providing a structured and systematic validation approach using "validation obligations." Validation obligations (VOs) help in checking the compliance and suitability of a specification with

[1]Many definitions of validation exist; in the context of this paper, we use the definition from [6]: "...determine whether an item fits the user needs, in particular with respect to safety and quality and with emphasis on the suitability of its operation in accordance to its purpose in its intended environment."

[2]A PO is a logical formula associated with the correctness claim of a given verification property.

978-1-6654-0140-1/20/$31.00 ©2021 IEEE
DOI 10.1109/ICSE-NIER52604.2021.00009

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 23,2021 at 23:14:11 UTC from IEEE Xplore. Restrictions apply.

respect to the stakeholders' requirements, and also by providing support for incremental modeling and model evolution. While classical refinement used for code generation has a *single* linear refinement chain, the VOs methodology allows *multiple abstract views* of the same system (for multiple stakeholders depending upon their technical skills), and also *multiple instantiations* of a generic component for testing and simulation purposes as high-level acceptance tests can also be modeled as VOs, supported by refinement concepts.

While POs ensure the internal consistency of a formal model and guarantee correct refinement, VOs ensure that the formal model correctly captures elicited requirements, help discover missing requirements, and help stakeholders track the evolution of their requirements and uncover high-level violations after model evolution. This is built around a simple observation: step-wise formal refinement is a well-established, effective method to master the intrinsic complexity of verification, i.e., checking the consistency and well-definedess of an implementation – or an incremental change thereof – against its specification. As the validation of a system's implementation against its requirements is likewise challenging, it is expected that the step-wise refinement process – breaking the task into smaller validation steps – will enable this task to be mastered more easily and systematically. It is crucial that each refinement step preserves the validations already carried out at the higher abstraction levels.

Rest of the paper is structured as follows: Section II provides the necessary background to understand the content of this paper. Section III presents the methodology used to achieve validation obligations. Section IV presents results of the feasibility study conducted using the proposed approach. The paper is concluded in Section V with the proposed future work.

II. Background

a) Formal specification and refinement: Many techniques, methods, and development processes have been proposed to assist developers in constructing reliable software. We believe that state-based rigorous methods [9] are the most promising for the development of safety-critical software. They support the "correct by construction" paradigm through the use of mathematically-based specification languages and the notion of refinement. While in this work we use Event-B and its ecosystem as the candidate formal method, our proposed approach is generally applicable to state- and refinement-based rigorous methods and is not only limited to Event-B.

b) Validation: Validation requires a judgment to assess whether the work is getting nearer to fulfill the needs of the ultimate customers. Validation demands that stakeholders be convinced by evidences. The techniques must allow the people who make the validation to imagine how the artifact will work out in its environment. In traditional engineering, different sets of tools are used to validate artifacts: blueprints, mock-ups, prototypes, or simulations, for instance. Software engineers have fewer tools at their disposal though. In a typical software development, the requirements in the beginning phase differ from the ones in the ending phase [10]. Two reasons account for this:

- requirements are evolving: as the development progresses, users, consumers, and stakeholders get a better understanding of the impact of the future artifact on their environment. As a consequence, they will discover new needs or mis-adaptations;
- requirements are incomplete: refinement requires the developers to make decisions on how to represent abstract structures or how to decompose actions. Often, such decisions cannot be derived from the requirements; they introduce new "features" into the software.

The appeal and success of Agile methodologies [11] come from this issue about managing requirements' evolution. The development process promoted by those methodologies is based on simple principles:

1) split the development into incremental *runs,*
2) each run must be short,
3) each run must lead to a testable product, and
4) the last run must produce the complete implementation.

The third step improves the odds that an anomaly or an incompleteness in the requirements will be detected as soon as possible. Since the seminal work of Boehm [12], we know that the earlier an anomaly is detected, the cheaper it is to correct.

The VOs approach enables a more agile-like methodology to be used for the development of safety-critical systems: it allows one to determine the effects of requirements changes and easily adapt a formal model in such a way that important requirements keep being satisfied.

As far as validation is concerned, various works have studied how linear temporal logic (LTL) properties are affected by refinement in the context of Event-B [13] and Z [14]. These works provide foundations to optimize the checking of VOs related to model checking. The approaches based on the fusion of the goal-oriented requirements engineering method: keep all objectives satisfied (KAOS) and Event-B, such as [15], are also relevant in this context as they provide the logical foundations of our work and act as its precursors.

c) Tools and formal issues about validation: A formal model can be executed (for validation purposes) through one of these three techniques: translation, animation, or simulation.

In principle, through refinements, by gradually lowering non-determinism, we move towards fully deterministic models, which can be easily transformed into programs. This is what the translators like EB2ALL [16] or Asm2C++ [17] do. However, both translators pose strong restrictions on the models they are able to translate such as determinism (only one event enabled at each time), implementable data structures (no sets or quantified expressions), and substitutions as assignments.

The need for executing non-deterministic formal models has been recognized since beginning and some tools have also been provided. While Brama [18] is a pure animator,

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 23,2021 at 23:14:11 UTC from IEEE Xplore. Restrictions apply.

ProB [19] – a model checker – can also be used to run animations. Animators are based on implementations of the operational semantics of the language. They can deal with some non-determinism such as choosing events to fire or choosing values for event-parameters. While they impose less constraints on specifications than translators, they still put some strong limits on the class of models that can be animated. To this end, techniques like VTA (verify-transform-animate) [20], which are based on syntactic transformations of specifications can help render non-animatable specifications animatable up to certain extent. Of course, the correctness of such transformations becomes a crucial issue then.

Translator and animators may fail. Then a technique called *simulation* [21] may work. The idea is to combine translators (by generating executable implementations of formal traits), animators (by providing an engine and libraries implementing the operational semantics), and users' intelligence (by providing them with hooks where they can insert their own code to overcome non-determinism or to prevent combinatorial explosions). The simulators may not exhibit all the behaviors specified, but the judicious restriction to the "interesting" behaviors allows validation to take place.

d) ProB: ProB is a model checking software, which also provides animation capabilities. Initially, ProB was developed as a model checker for the B language. It was extended with animation facilities, which allow developers to run interactive and graphical executions of formal models. ProB now supports a variety of state-based formal methods. ProB provides formal methods users with functionalities such as deadlock checking, disproving (finding counter examples in proofs), or animation. Our proposed approach is built upon the ProB platform.

III. Validation Obligations

Our methodology to realize the VOs approach first consists in characterizing and formalizing the notion of VOs. In particular, we classify how to discharge VOs and how they are affected by refinement and by incremental model changes. Some VOs may require human intervention, e.g., to evaluate a graphical rendering of the state-space or of a particular state as shown in Fig. 1; here a stakeholder may wish to inspect that the blinking behavior of the vehicle corresponds to his/her expectations.

Other VOs, like a temporal LTL property as shown in Fig. 2, can be discharged automatically by model checking and when proven for an abstraction may hold for a refinement or only require a simplified additional check to be carried out. While some VOs may require further instantiations of the model, others, on the contrary, may require further abstractions.

In order to organize VOs, we hence propose to use non-linear refinement chains. In Fig. 3, M0, M1, M2, and Impl correspond to the classical linear refinement chain, where M0 is the most abstract and Impl the most concrete model. The models M2a and M2b, however, are instantiations of M2 for validation purposes, while A1 and A2 are additional abstract views, e.g., projections of the system for various stakeholders' concerns. Possible VOs here are:

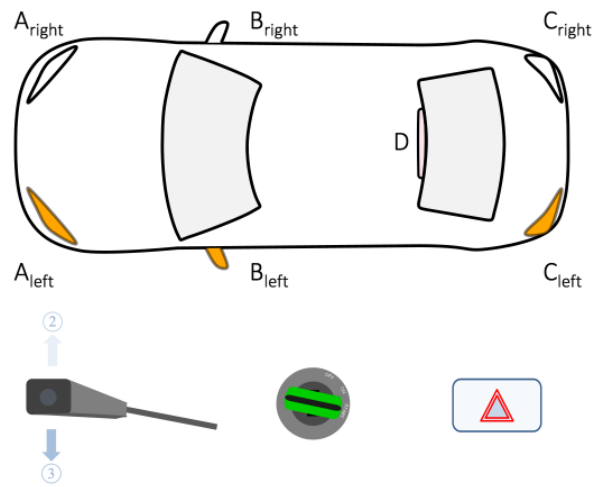

Fig. 1. Graphical visualization of the state of the adaptive exterior vehicle lighting model in the prototype

Machine	Model Checki...	LTL
GenericTimers	?	?
Sensors	✓	?
PitmanController_TIME_MC_v4	✓	✓
PitmanController_TIME_v4	?	?
BlinkLamps_v3	✓	✓
PitmanController_v6	✓	✓

Fig. 2. Project view of the prototype UI related to model checking and LTL properties checking

- use cases expressed as state machines or high-level assurance tests,
- collections of detailed scenarios (see Fig. 4),
- concrete unit tests,
- temporal properties checking functional adequacy,
- coverage properties (MC/DC) to ensure that all parts of the model are necessary and validated,
- tests run on the final generated code, etc.

Similarly, an execution trace for a refinement (e.g., U1 on the right side of Fig. 3) can be translated into an execution trace of an abstract view of the system.

Regarding nonlinear refinement for validation, we need to capture the fact that there can be multiple views, e.g., multiple abstractions for the same system or component. We capture multiple instantiations of generic components for validation purposes in our refinement structure. This gives rise to nonlinear refinement lattices (cf., Fig. 3) rather than one linear refinement chain. Examples of nonlinear refinement are instantiation refinement or alternate view refinement as shown in Fig. 3. M2 in Fig. 3 is, in general, not the composition of M1, A1 and A2; A1 and A2 are alternate (stakeholder) views of the system and may overlap in ways not allowed by formal decomposition.

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 23,2021 at 23:14:11 UTC from IEEE Xplore. Restrictions apply.

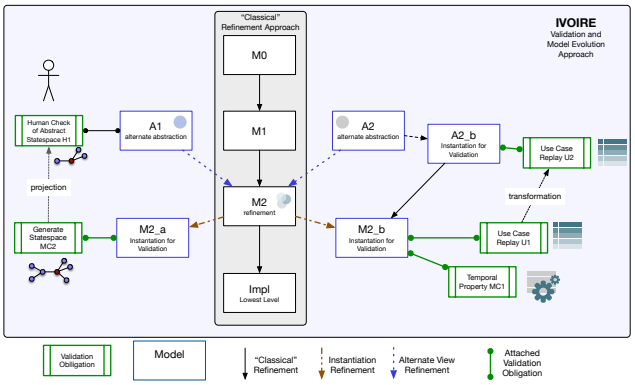

Fig. 3. Nonlinear refinement sketch

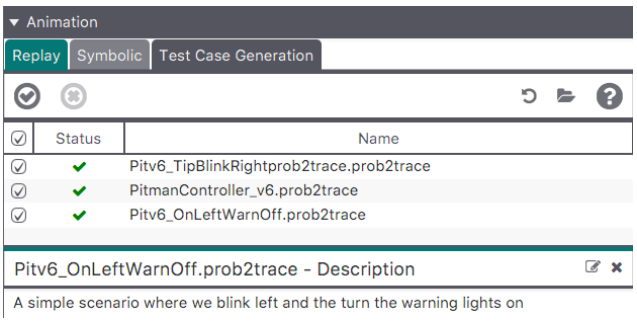

Fig. 4. Stored traces/scenarios in the prototype UI

IV. Feasibility Study

We test the feasibility of our approach on two case studies:

1) The first case study is about the development of a formal specification based on the new hybrid level 3 European train control system (ETCS) principles document [22]. The VOs-inspired development leads the uncovering of around 40 issues in the original requirements document, leading to changes both to the original requirements document (updated versions of [22] are now available) as well as to our initial formal model. The visualizations of the formal model (see Fig. 5) were extremely useful to allow domain experts to understand and correct the issues. Managing the model evolution using the current state-of-the-art, i.e., without VOs, was very cumbersome and error prone.
2) The second case study is about the formal specification of an adaptive exterior light and speed control system [23]. In this VOs-inspired formal development, before checking POs, we validated our models using model checking and animation with visualizations as shown in Fig. 1 and Fig. 2. This not only helped uncover requirements errors but also helped transforming the model into an executable, interactive reference specification, which could also be examined by users without any formal background. The model checking and LTL formulas checking further helped ensuring the overall validation of the specification by breaking it into smaller steps associated with each refinement.

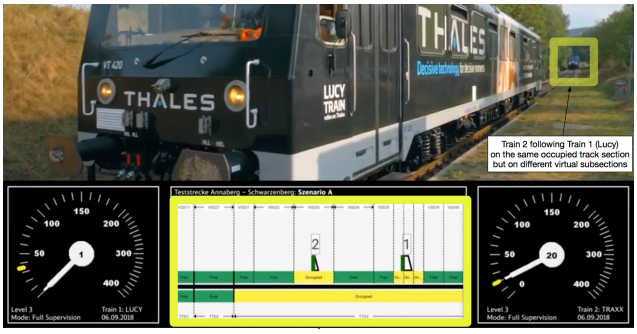

Fig. 5. Screen-shot from a video of Deutsche Bahn showing a formal model in action https://www.youtube.com/watch?v=K6mS6akRmvA

V. Conclusion

In this paper, we have proposed the notion of VOs to formally check the compliance between requirements and their formal specification. The presented approach is also useful in facilitating incremental modeling and tracking model evolution. We have checked the feasibility of the proposed approach on real-life industrial-strength case studies of ETCS HL 3 and the adaptive exterior light and speed control system of a vehicle.

Tooling is an important part of the formal methods research, both to help validate the developments and to disseminate them. To this end, we have developed a prototype (an extension of ProB), which is able to design and validate scenarios in the form of specification animation, generate and execute test cases, and perform model checking and LTL properties checking. In fact, the prototyped VOs manager keeps track of the discharged VOs, model evolution, and the information that how various VOs are linked to refinement (as shown in Fig. 2 and Fig. 4).

In the future, we intend to further develop the tool as hinted in Sec. III. We also intend to conduct more case studies and controlled experiments as a part of the future work.

Data Availability

The ProB tool used in this research is openly available at https://prob.hhu.de.

Acknowledgment

The research reported in this paper has been partly funded by the Austrian Science Fund (FWF) (grant # I 4744-N), and the LIT Secure and Correct System Lab sponsored by the province of Upper Austria.

References

[1] J. Rushby, "Formal Methods and the Certification of Critical Systems," Computer Science Laboratory – SRI International, Technical Report CLS-93-7, December 1993.

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 23,2021 at 23:14:11 UTC from IEEE Xplore. Restrictions apply.

[2] J.-R. Abrial, M. Butler, S. Hallerstede, T. S. Hoang, F. Mehta, and L. Voisin, "Rodin: an open toolset for modelling and reasoning in Event-B," *International Journal on Software Tools for Technology Transfer*, vol. 12, no. 6, pp. 447–466, 2010.

[3] J. Jacquot and A. Mashkoor, "The role of validation in refinement-based formal software development," in *Models: Concepts, Theory, Logic, Reasoning and Semantics - Essays Dedicated to Klaus-Dieter Schewe on the Occasion of his 60th Birthday*, 2018, pp. 202–219.

[4] N. G. Leveson, M. P. E. Heimdahl, and J. D. Reese, "Designing specification languages for process control systems: Lessons learned and steps to the future?" in *Software Engineering — ESEC/FSE '99*, O. Nierstrasz and M. Lemoine, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 127–146.

[5] C. Heitmeyer, "SCR: a practical method for requirements specification," in *17th DASC. AIAA/IEEE/SAE. Digital Avionics Systems Conference. Proceedings (Cat. No.98CH36267)*, vol. 1, 1998, pp. C44/1–C44/5 vol.1.

[6] CENELEC, "Railway applications – communication, signalling and processing systems – software for railway control and protection systems," European Standard, Tech. Rep. EN50128, 2011.

[7] J.-R. Abrial, *The B Book*. Cambridge University Press, 1996.

[8] ——, *Modeling in Event-B: System and Software Engineering*. Cambridge University Press, 2010.

[9] A. Mashkoor, F. Kossak, and A. Egyed, "Evaluating the suitability of state-based formal methods for industrial deployment," *Software: Practice and Experience*, vol. 48, no. 12, pp. 2350–2379, 2018. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.2634

[10] T. Nakatani, T. Tsumaki, M. Tsuda, M. Inoki, S. Hori, and K. Katamine, "Requirements maturation analysis by accessibility and stability," in *Software Engineering Conference (APSEC), 2011 18th Asia Pacific*, 2011, pp. 357–364.

[11] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ, USA: Prentice Hall PTR, 2003.

[12] B. W. Boehm, *Software Engineering Economics*, 1st ed. Upper Saddle River, NJ, USA: Prentice Hall PTR, 1981.

[13] T. S. Hoang, S. Schneider, H. Treharne, and D. M. Williams, "Foundations for using linear temporal logic in Event-B refinement," *Formal Asp. Comput.*, vol. 28, no. 6, pp. 909–935, 2016. [Online]. Available: http://dx.doi.org/10.1007/s00165-016-0376-0

[14] J. Derrick and G. Smith, "Linear temporal logic and Z refinement." in *AMAST'04*, 2004, pp. 117–131.

[15] A. Mashkoor and A. Matoussi, "Towards validation of requirements models," in *Abstract State Machines, Alloy, B and Z, Second International Conference, ABZ 2010, Orford, QC, Canada, February 22-25, 2010. Proceedings*, 2010, p. 404. [Online]. Available: https://doi.org/10.1007/978-3-642-11811-1_38

[16] N. K. Singh, "EB2ALL: An Automatic Code Generation Tool," in *Using Event-B for Critical Device Software Systems*. Springer London, 2013, pp. 105–141.

[17] S. Bonfanti, A. Gargantini, and A. Mashkoor, "Design and validation of a C++ code generator from abstract state machines specifications," *J. Softw. Evol. Process.*, vol. 32, no. 2, 2020. [Online]. Available: https://doi.org/10.1002/smr.2205

[18] T. Servat, "BRAMA: A New Graphic Animation Tool for B Models," in *B'2007: Formal Specification and Development in B*. Springer-Verlag, 2006, pp. 274–276.

[19] M. Leuschel and M. Butler, "ProB: An Automated Analysis Toolset for the B Method," *Journal Software Tools for Technology Transfer*, vol. 10, no. 2, pp. 185–203, 2008.

[20] A. Mashkoor and J.-P. Jacquot, "Validation of formal specifications through transformation and animation," *Requirements Engineering*, vol. 22, no. 4, pp. 433–451, Nov 2017. [Online]. Available: https://doi.org/10.1007/s00766-016-0246-6

[21] A. Mashkoor, F. Yang, and J.-P. Jacquot, "Refinement-based Validation of Event-B Specifications," *Software & Systems Modeling*, vol. 16, no. 3, pp. 789–808, 2017. [Online]. Available: http://dx.doi.org/10.1007/s10270-016-0514-4

[22] "Hybrid ERTMS/ETCS Level 3," EEIG ERTMS Users Group, 123-133 Rue Froissart, 1040 Brussels, Belgium, Principles Ref: 16E042, Version: 1A, 7 2017.

[23] F. Houdek and A. Raschke, "Adaptive exterior light and speed control system," in *Rigorous State-Based Methods*, A. Raschke, D. Méry, and F. Houdek, Eds. Cham: Springer International Publishing, 2020, pp. 281–301.

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 23,2021 at 23:14:11 UTC from IEEE Xplore. Restrictions apply.